

2024 年臺灣國際科學展覽會

研究報告

區別：

科別：電腦科學與資訊工程科

作品名稱：探討以卷積神經網路辨識人工或 AI 生成圖片之研究

關鍵詞：CNN、AI 圖片、辨識生成

編號：

摘要

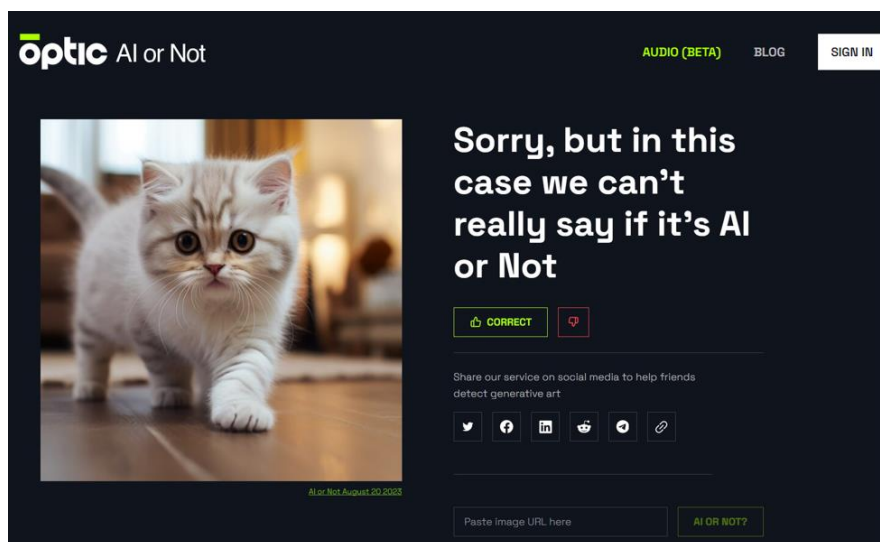
隨著圖片生成技術的提升，使原本利用可視化噪音判斷是否為 AI 生成圖片的平台準確率下降。因此，本研究目的為利用卷積神經網路，藉由判斷不可視化噪音，來辨識是否為 AI 生成圖片。實驗主要分為三個部分：分別透過 L1、L2 正則化與調整動態學習率進行訓練，再將正則化與動態學習率同時添加到卷積神經網路上尋找最佳參數，最後與現有的 AI 辨識平台進行比較。

Abstract

With the advancement of image generation techniques, the accuracy of platforms that previously relied on visualizing noise to determine whether an image was generated by AI has declined. Therefore, the purpose of this study is to utilize convolutional neural networks to discern AI-generated images by identifying non-visualized noise. The experiment is divided into three main parts: training using L1 and L2 regularization and dynamic learning rates separately, then simultaneously adding regularization and dynamic learning rates to the convolutional neural network to find the optimal parameters, and finally comparing with existing AI recognition platforms.

壹、研究動機

隨著圖片生成技術的提升，近年來越來越多圖片生成平台像是 Stable Diffusion 與 MidJourney 等，都可以生成出讓肉眼無法分辨真偽的圖片。雖然現階段有藉由可視化噪聲判斷 AI 生成圖片的平台像是 Optic AI or Not，但隨著生成技術的提升，可視化噪聲越來越難分辨，判斷生成圖片的平台準確度降低。因此我們想藉由卷積神經網路，判斷不可視噪聲，來增加準確度。



圖一、判斷 AI 生成圖片的 Optic AI or Not 平台

貳、研究目的

目前辨識圖片真偽平台，多透過可視化噪聲判斷是否為 AI 生成圖片，然而有許多不可視化噪聲是肉眼無法分辨的。因此本實驗透過卷積神經網路，判斷圖片中不可視化噪聲，來判斷是否為 AI 生成圖片，並與現有的 AI 圖片判斷網站進行比較。本作品主要有三點目的：

1. 研究卷積神經網路模型判斷 AI 與人工圖片
2. 改變正則化參數及動態學習率找出最佳模型
3. 與現有 AI 圖片辨別網站進行比較

參、研究過程與方法

(一) 設備與環境：

1. 硬體設備：

記憶體 16GB

CPU: Intel Core i5-4440 3.1GHz

GPU: NVIDIA GeForce RTX 3060

2. 軟體環境：

Python: anaconda3

深度學習框架: tensorflow

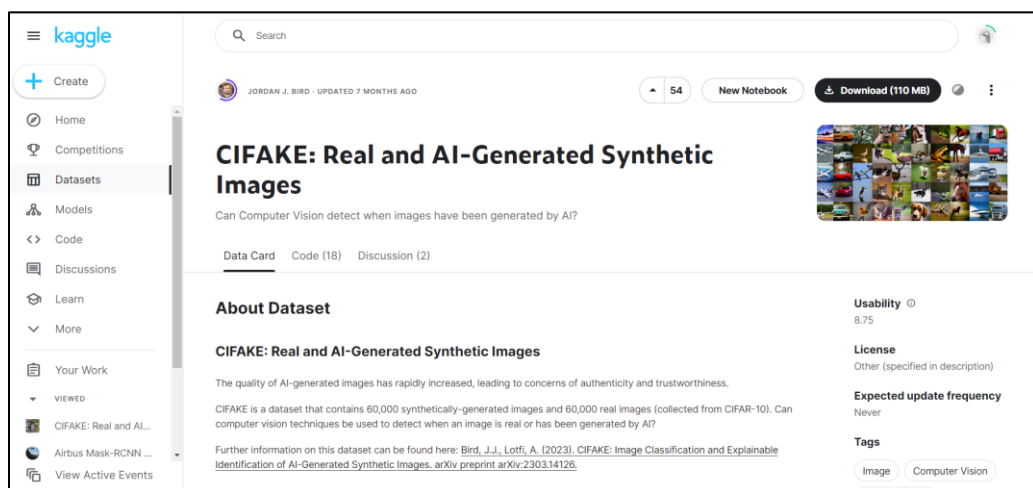
函式庫: Keras、numpy、cv2、matplotlib

(二) 研究流程：

1. 資料蒐集: CIFake 資料集(Real and AI-Generated Synthetic images)

CIFake 資料集是一個包含兩個標籤，合成影像和真實影像，並由 60000 張合成影像和 60000 張真實影像組合成的資料集，其中 60000 張真實影像是從 CIFAR-10 收集，再透過 Stable Diffusion version 1.4 生成出另外 60000 張合成影像。

CIFake 資料集是從 Kaggle 平台上獲取，雖然總共有 120000 張影像，但因硬體內存不足，此次研究只使用合成與真實影像各 5000 張，總共 10000 張來進行訓練。



圖二、Kaggle 平台上的 CIFake 資料集[2]

2. 圖片預處理

本研究為了得到更高分辨率來顯示更細微的特徵圖案，因此將原本 32*32 影像 resize 成 80*80。之後進行歸一化處理，將圖像每個像素值除以 255 使像素值落於 0 到 1 之間，使訓練加速收斂。

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

歸一化公式(x 是原始特徵數據， x_{norm} 是歸一化後的數據再 $[0, 1]$ 之間， x_{min} 和 x_{max} 分別是數據特徵的最小值和最大值。

3. 建立圖片辨識模型: VGG16 模型

本次研究採用 VGG16 模型，並在第一層卷積層與全連接層加入正則化處理，以下是模型結構:[1]

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 80, 80, 64)	1792
activation (Activation)	(None, 80, 80, 64)	0
conv2d_1 (Conv2D)	(None, 80, 80, 64)	36928
activation_1 (Activation)	(None, 80, 80, 64)	0
max_pooling2d (MaxPooling2D)	(None, 40, 40, 64)	0
conv2d_2 (Conv2D)	(None, 40, 40, 128)	73856
activation_2 (Activation)	(None, 40, 40, 128)	0
conv2d_3 (Conv2D)	(None, 40, 40, 128)	147584
activation_3 (Activation)	(None, 40, 40, 128)	0
max_pooling2d_1 (MaxPooling2D)	(None, 20, 20, 128)	0
conv2d_4 (Conv2D)	(None, 20, 20, 256)	295168
activation_4 (Activation)	(None, 20, 20, 256)	0
conv2d_5 (Conv2D)	(None, 20, 20, 256)	590080
activation_5 (Activation)	(None, 20, 20, 256)	0
conv2d_6 (Conv2D)	(None, 20, 20, 256)	590080
activation_6 (Activation)	(None, 20, 20, 256)	0
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 256)	0
conv2d_7 (Conv2D)	(None, 10, 10, 512)	1180160
activation_7 (Activation)	(None, 10, 10, 512)	0
conv2d_8 (Conv2D)	(None, 10, 10, 512)	2359808
activation_8 (Activation)	(None, 10, 10, 512)	0
conv2d_9 (Conv2D)	(None, 10, 10, 512)	2359808
activation_9 (Activation)	(None, 10, 10, 512)	0
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 512)	0
conv2d_10 (Conv2D)	(None, 5, 5, 512)	2359808

activation_10 (Activation)	(None, 5, 5, 512)	0
conv2d_11 (Conv2D)	(None, 5, 5, 512)	2359808
activation_11 (Activation)	(None, 5, 5, 512)	0
conv2d_12 (Conv2D)	(None, 5, 5, 512)	2359808
activation_12 (Activation)	(None, 5, 5, 512)	0
max_pooling2d_4 (MaxPooling2D)	(None, 2, 2, 512)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 4096)	8392704
activation_13 (Activation)	(None, 4096)	0
dense_1 (Dense)	(None, 4096)	16781312
activation_14 (Activation)	(None, 4096)	0
dense_2 (Dense)	(None, 2)	8194
activation_15 (Activation)	(None, 2)	0

表一、VGG16 結構

4. 正則化

為了避免無效的學習導致時間浪費，我們增加了 EarlyStopping 的 Callbacks 參數，並將 patience 設為 10 當使 val_loss 的值連續 10 輪沒有下降時，強制結束訓練。

```
early_stopping_callback = EarlyStopping(monitor='val_loss',
                                         patience=10,
                                         verbose=1,
                                         restore_best_weights=True)
```

圖三、EarlyStopping Callback 參數

此外，為了避免過擬合問題，我們在 VGG16 模型的第一層卷積層與全連接層中，加入 L1 正則化與 L2 正則化來避免其過擬合。

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N |W_i|$$

L1 正則化公式

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N W_i^2$$

L2 正則化公式

其中 y 為實際值， $\hat{y}$ 為預測值， λ 為正則化參數， W 為模型中的權重[3]

5. 動態學習率

為了找到最佳的模型參數，我們藉由自適應矩估計(Adam)結合學習率回調(ReduceLROnPlateau 及 Learning_rate_scheduler)來尋找最佳學習率。

Adam 為一種自適應學習率優化算法，並結合了 RMSprop 和動量梯度下降算法，而 ReduceLROnPlateau 為 Keras 內建方法，其中 patience=5，當 val_loss 未下降且步長超過 5 時，lr*factor 得到新的 lr，最小 lr 為 min_lr。

```
reduce_lr = ReduceLROnPlateau(monitor='val_loss',  
                               factor=0.2,  
                               patience=5,  
                               min_lr=0.00001)
```

圖四、ReduceLROnPlateau 參數設定[4]

Learning_rate_scheduler 也是 Keras 內建方法，可透過此函數來自定義動態學習率。每當步長超過 5 時，lr=lr*0.1 藉此來逐漸降低學習率，達到更好的收斂效果。

```
def scheduler(epoch, lr):  
    if epoch % 5 == 0 and epoch != 0:  
        return lr * 0.1  
    else:  
        return lr  
learning_rate_scheduler = LearningRateScheduler(scheduler)
```

圖五、Learning_rate_scheduler 參數設定[5]

6. 二元交叉熵 (Binary cross-entropy)

二元交叉熵損失通常用於二元分類問題，即將輸入分類為兩個類別之一（合成圖片與真實圖片）。它衡量預測概率和真實標籤之間的差異。二元交叉熵損失的公式如下：

$$L = \sum_i y_i \log O_i + (1 - y_i) \log(1 - O_i)$$

其中 y 表示真實標籤（0 或 1）， O_i 表示正類別的預測概率

肆、研究結果與討論

一、VGG16 模型對 CIFake 資料集訓練結果

(一) 單純以 VGG16 模型進行訓練

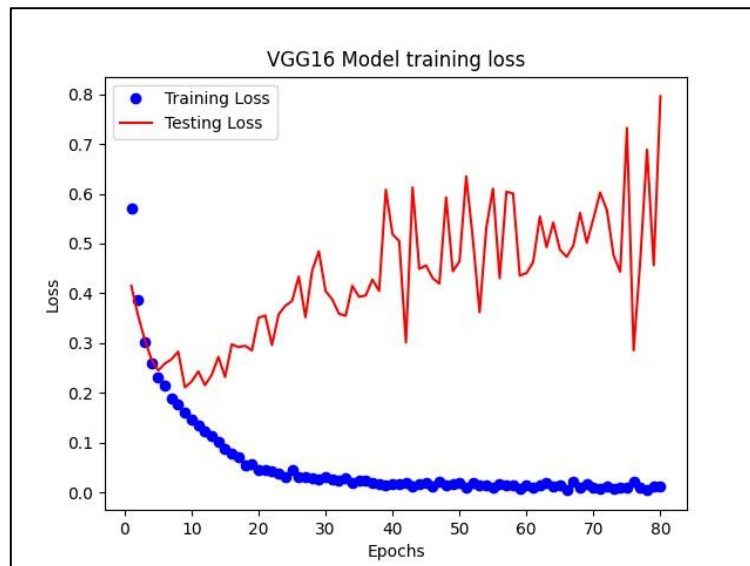
1. 實驗目的

作為增加正則化與 Callbacks 參數前對照組。

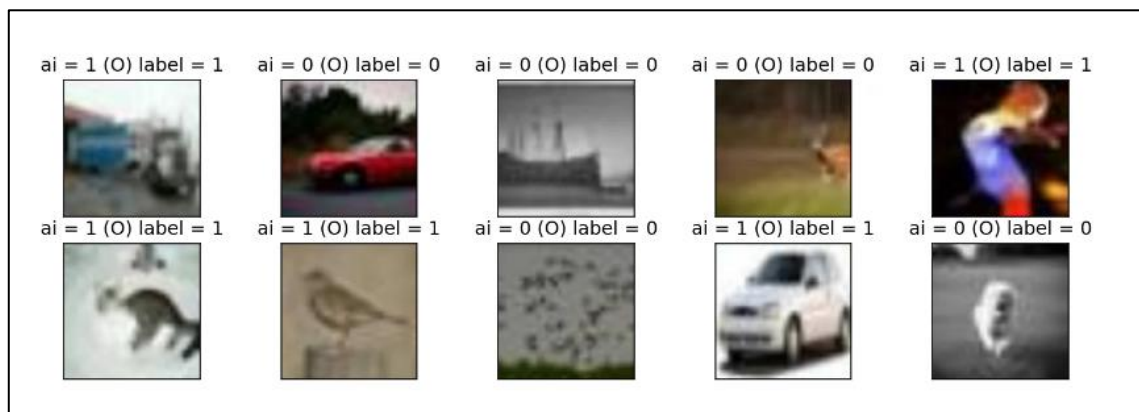
2. 參數設定

將輸入層尺寸改為 80*80(原 225*225)以符合本實驗要求，epoch = 80，未使用 Callback 函數。

3. 實驗結果



圖六、VGG16 模型對 CIFake 資料集訓練結果



圖七、輸出預測結果前十項範例

4. 結果討論

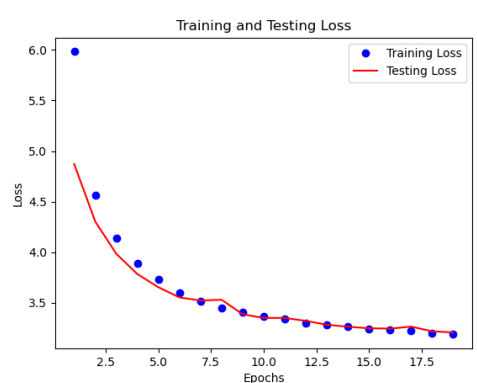
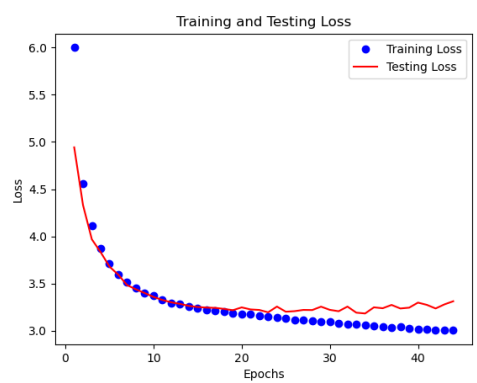
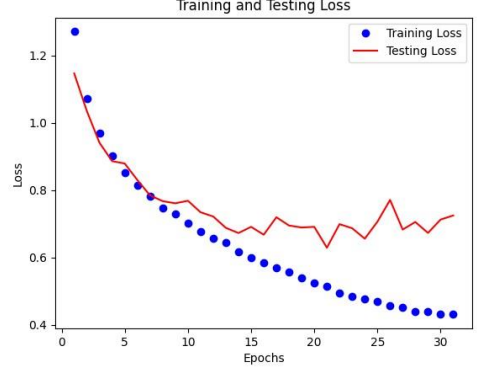
在沒有正則化及動態學習率的幫助下，模型有過擬合的現象發生，且在 epoch=5 的地方 testing loss 就有上升的趨勢。

二、正則化與動態學習率

(一) 改變正則化

1. 正則化在機器學習和深度學習中被用來解決過度擬合 (overfitting) 的問題，而 L1 和 L2 正則化是最常見的正則化技術之一。它們通過對損失函數添加懲罰項來限制模型的權重，進而減少過度擬合。因為訓練前期的震盪較大，loss 值也較大，因此底下圖表皆為 epoch=2 開始繪製，下表為只有 L1L2 正則化之結果。

2. 參數設定: 輸入尺寸(80, 80, 3), lr=0.0001, batch size =32:

Num	Early stopping epochs	L1L2	Testing accuracy Testing loss	Loss-epoch chart
1	Epoch = 20	L1=0.01 L2=0.01	Testing accuracy = 0.9207 Testing loss = 3.206	
2	Epoch = 45	L1=0.01 L2=0.0001	Testing accuracy = 0.9210 Testing loss = 3.181	
3	Epoch = 32	L1=0.001 L2=0.001	Testing accuracy = 0.9265 Testing loss = 0.6196	

4	Epoch = 35	L1=0.001 L2=0.0001	Testing accuracy = 0.9300 Testing loss = 0.6066	
5	Epoch = 25	L1=0.0001 L2=0.0001	Testing accuracy = 0.9327 Testing loss = 0.2602	

3. 結果討論

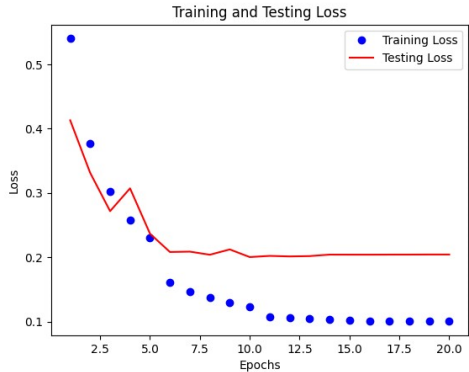
當 L1 與 L2 下降時，testing accuracy 越高，雖然有稍微過擬合的現象，但因為 testing loss 與 training loss 的值差距不大，因此我們認為還在可接受的範圍內。

(二) 改變動態學習率

1. 動態學習率能夠自動調整學習率，通常在訓練的不同階段控制模型的收斂速度。這樣做的原因是在訓練過程中，如果學習率過大，則可能會錯過最優解，如果學習率過小，則收斂速度會很慢。因此，動態調整學習率能夠幫助模型更好地收斂，下表為只有 Callback 函數之結果。

2. 參數設定：輸入尺寸(80, 80, 3), lr=0.0001, batch size =32:

Num	Early stopping epochs	Callback	Testing accuracy Testing loss	Loss-epoch chart
6	Epoch = 23	scheduler	Testing accuracy = 0.9185 Testing loss = 0.2268	

7	Epoch = 20	Reduce_lr	Testing accuracy = 0.9292 Testing loss = 0.1868	
---	------------	-----------	--	--

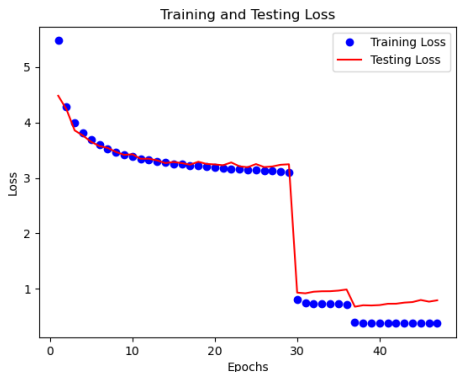
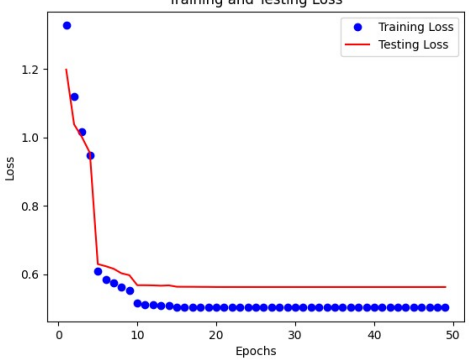
3. 結果討論:

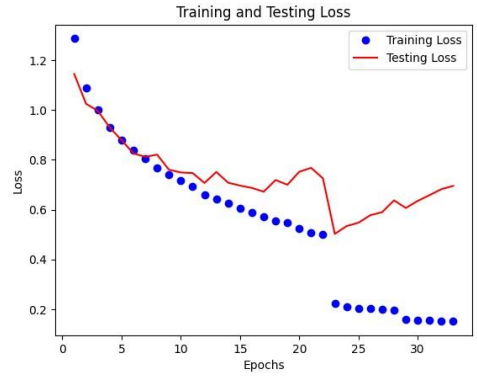
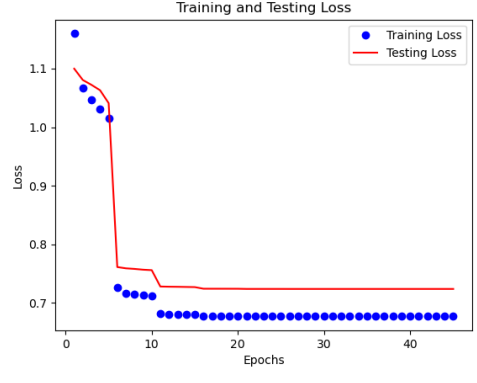
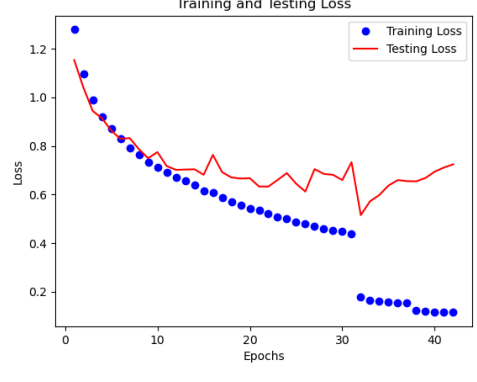
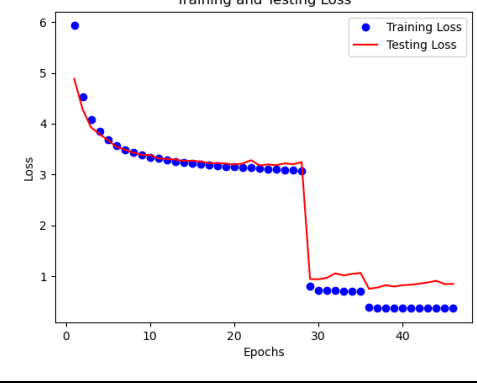
從上面兩個圖表可知，增加 Callback 函數有稍微降低原本 VGG16 的 loss 值，且 reduce_lr 的效果較好。從 num6 的圖可知，當 epoch=7 時 testing loss 就有上升的趨勢，原因是步長每 5 步 lr 才下降一次，沒有 Reduce_lr 下降來的即時。

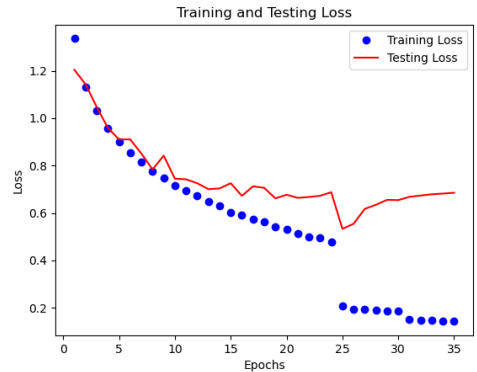
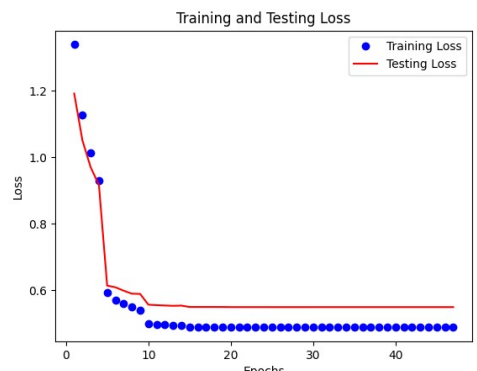
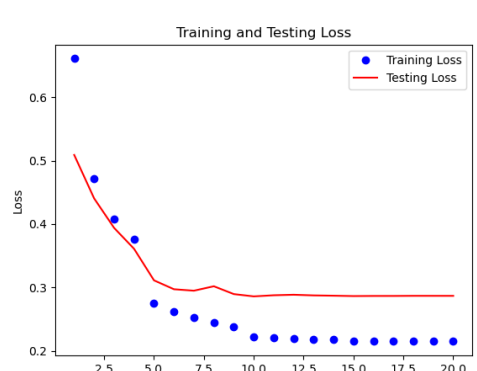
(三) 改變正則化與動態學習率

1. 從上面兩個圖表可知正則化與動態學習率可降低模型過擬和程度。因此，透過正則化防止過度擬合，並透過動態學習率提高模型的訓練速度和性能，找出最適當的模型。下表為改變正則化與動態學習率的結果。

2. 參數設定: 輸入尺寸(80, 80, 3), lr=0.0001, batch size =32:

Num	Early stopping epochs	L1L2 CallBack	Testing accuracy Testing loss	Loss-epoch chart
8	Epoch = 48	L1=0.01 L2=0.01 Reduce_lr	Testing accuracy = 0.9277 Testing loss = 0.6946	
9	Epoch = 50	L1=0.01 L2=0.01 scheduler	Testing accuracy = 0.9287 Testing loss = 0.5490	

10	Epoch = 34	L1=0.01 L2=0.0001 Reduce_lr	Testing accuracy = 0.9312 Testing loss = 0.4769	 Training and Testing Loss Y-axis: Loss (0.2 to 1.2). X-axis: Epochs (0 to 35). Legend: Training Loss (blue dots), Testing Loss (red line).
11	Epoch = 50	L1=0.01 L2=0.0001 scheduler	Testing accuracy = 0.9112 Testing loss = 0.7089	 Training and Testing Loss Y-axis: Loss (0.7 to 1.1). X-axis: Epochs (0 to 45). Legend: Training Loss (blue dots), Testing Loss (red line).
12	Epoch = 43	L1=0.001 L2=0.001 Reduce_lr	Testing accuracy = 0.9369 Testing loss = 0.4741	 Training and Testing Loss Y-axis: Loss (0.2 to 1.2). X-axis: Epochs (0 to 45). Legend: Training Loss (blue dots), Testing Loss (red line).
13	Epoch = 47	L1=0.001 L2=0.001 scheduler	Testing accuracy = 0.9367 Testing loss = 0.7016	 Training and Testing Loss Y-axis: Loss (1 to 6). X-axis: Epochs (0 to 45). Legend: Training Loss (blue dots), Testing Loss (red line).

14	Epoch = 36	L1=0.001 L2=0.0001 Reduce_lr	Testing accuracy = 0.9359 Testing loss = 0.4923	
15	Epoch = 48	L1=0.001 L2=0.0001 scheduler	Testing accuracy = 0.9294 Testing loss = 0.5357	
16	Epoch = 22	L1=0.0001 L2=0.0001 Reduce_lr	Testing accuracy = 0.9272 Testing loss = 0.2821	
17	Epoch = 21	L1=0.0001 L2=0.0001 scheduler	Testing accuracy = 0.9309 Testing loss = 0.2699	

3. 結果討論:

雖然 num12 有最高的 testing accuracy，為 0.9369。然而其 loss 值相差較 num13 來的大，且 num13 圖的 testing accuracy 也有 0.9367，因此們最終選擇的參數設定為: L1=0.001, L2=0.001, CallBack = scheduler, Epochs=47。

三、探討自定義模型與現有 AI 圖片辨別網站比較

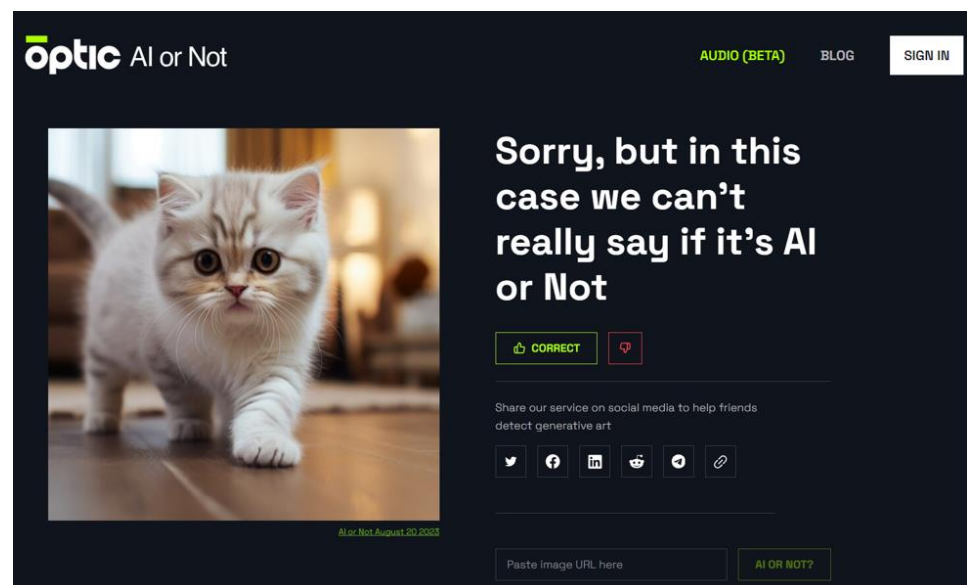
(一) 圖片選擇:

透過 midjourney 與 stable diffusion 公開圖片，尋找可視化噪音少，使 AI 圖片辨識平台無法正確辨識的圖片進行測試。圖為測試圖片，以下為各平台測試結果。



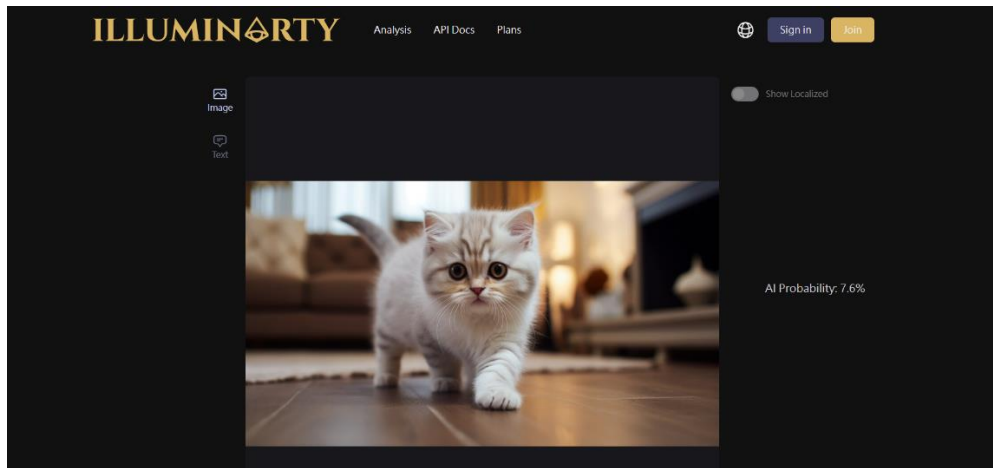
圖八、可使辨識平台辨識錯誤的 AI 生成圖片

1. Optic AI or Not



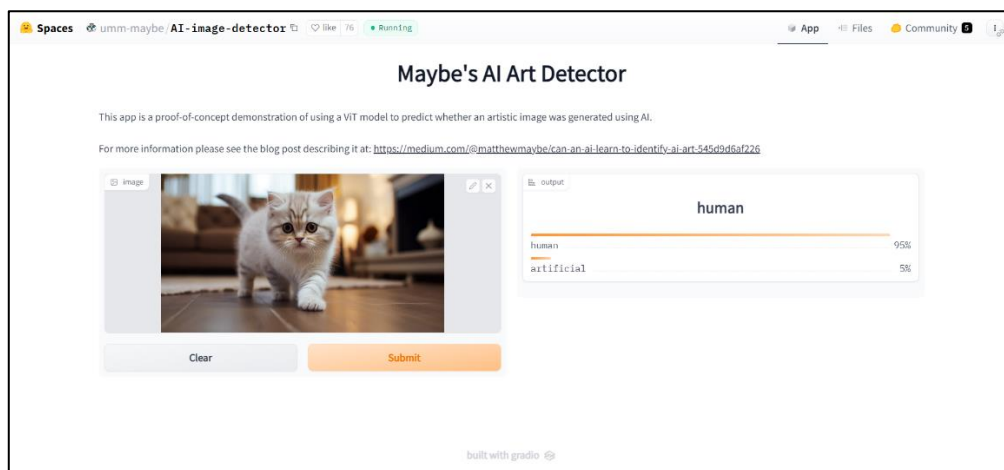
圖九、Optic AI or Not 無法正確識別

2. Illuminarty



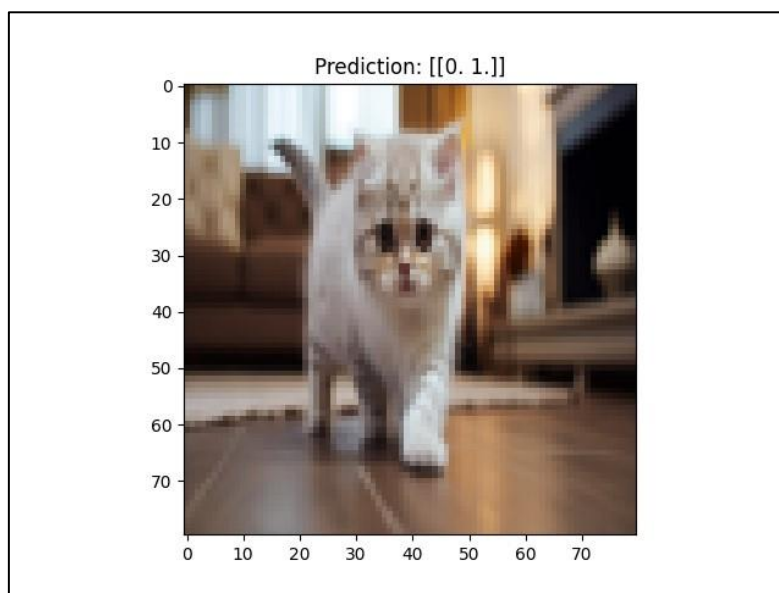
圖十、Illuminarty 認為只有 7.6% 為 AI 生成

3. Maybe's AI Art Detector



圖十一、Maybe's AI ART Detector 認為只有 5% 為 AI 生成

4. 自定義 VGG16 模型



圖十二、自定義 VGG16 模型成功辨識

伍、結論

本研究發現，在 VGG16 模型上增加正則化及動態學習率可以降低模型過擬合，並有效增加正確率。當參數設定為: L1=0.001, L2=0.001, CallBack = scheduler, Epochs=47 時，模型過擬合程度小，testing accuracy 高達 0.9367。此結果顯示模型有辨識不可視噪音之能力。

本研究透過與線上 AI 圖片辨識平台進行比較，並能夠成功辨識出其他平台無法辨識出的 AI 生成圖片。

陸、未來展望

(一) 嘗試其他模型

在 VGG Net 家族裡，有許多其他模型及衍伸模型像 VGG19 等，也有更新的模型希望可以在後續研究進行嘗試。

(二) 數據集優化

本作品採用的數據集每張圖片只有 32*32，使其準確率因細節的不足而無法提升。希望未來有機會以畫質更好的數據集進行研究。

柒、參考文獻資料

- [1] Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv preprint arXiv:1409.1556.
- [2] Birdy654. (n.d.). CIFAKE: Real and AI-Generated Synthetic Images. Kaggle.
<https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images/data>
- [3] Stack Overflow. (2019). Compute the Loss of L1 and L2 regularization. Retrieved from <https://stackoverflow.com/questions/58905671/compute-the-loss-of-l1-and-l2-regularization>
- [4] Keras. (n.d.). ReduceLROnPlateau. Keras API documentation. Retrieved from https://keras.io/api/callbacks/reduce_lr_on_plateau/
- [5] Keras. (n.d.). LearningRateScheduler. Keras API documentation. Retrieved from https://keras.io/api/callbacks/learning_rate_scheduler/